

Dijkstra and A*: teacher's guide

A 90-minute guided reconstruction

Central question: When do we know enough to stop looking for a cheaper route?

The class reconstructs successive algorithms through a repeatable pattern: propose a rule, predict the next step, expose a counterexample, identify the failed assumption, and repair the rule. Toeplitz and Lakatos motivate this design [P1, P2]; it is not a claim about the historical discovery order of these algorithms.

Audience: PhD students with basic graphs, Python, and asymptotic analysis. Prior familiarity with Dijkstra is useful: invite knowledgeable students to attack the claims and articulate proof obligations rather than asking them to pretend the algorithm is new.

Time	Block	Slides	Principal artifact
00-05	Applications and specification	1-3	Slides
05-09	Model versus storage	4	Graph + slide
09-16	Brute force	5-6	Board + checkpoint 01
16-28	Reconstruct and justify Dijkstra	7-10	Board + main graph
28-41	Lazy implementation	11-13	Checkpoints 02-04 + visualizer
41-48	Eager queue and complexity	14-15	Checkpoint 05 + queue visualizer
48-54	Why goal information matters	16-17	Main graph and comparison
54-68	A*, admissibility, reopening	18-21	Trace + four-node counterexample
68-77	A* implementation and analysis	22-23	Checkpoint 06 + visualizer
77-86	Comparative experiments	24	Notebook and saved results
86-90	Synthesis and exit exercise	25	Worksheet

Hard boundary: Start A* by minute 54. If behind, shorten the indexed-heap internals and the final experiments, not the Dijkstra proof or the reopening counterexample.

Before class: a five-minute technical rehearsal

Open `slides/dijkstra_astar.pptx`, `web/visualizer.html`, and `notebooks/lecture_solutions.ipynb`. The browser file works offline. No server or account is required. Python reference scripts need only Python 3.10+; Jupyter is optional and must already be installed if used.

Run these commands from the repository root:

```
python -m unittest discover -s tests -v
python -m algorithms.brute_force
python -m algorithms.dijkstra
python -m algorithms.dijkstra --eager
python -m algorithms.astar --reopening
```

Print the two-page student worksheet without its solutions. Keep the solution handout and this guide for yourself. Draw the main graph once at the left of the board. Reserve a narrow strip for the persistent record: **Claim | Failure | Repair**. Use the middle for distance tables and the right for the finalization proof.

Prepare two editor buffers: the active checkpoint and its solution. There is no benefit to debugging a mistyped import in front of the class. Every checkpoint has a runnable solution in `checkpoints/solutions/`.

Browser controls: Right/left arrows move between meaningful events. Shift+arrow moves one executed line. R resets. Space plays/pauses. Use the Jump to menu for the bookmarked stale/decrease/reopening moments. Use Next Event for teaching and line-step only at the exact operation students are questioning. A+ increases code size. The queue view can switch from priority order to the actual heap array.

Terminology to preserve: discovered = has a finite tentative label; expanded = outgoing edges processed; finalized = distance certified. A* expansion is not automatically finalization. The goal pop is counted as a pop, not an expansion.

Fallback: If Jupyter fails, use `python -m checkpoints.solutions.02_lazy_relaxation`, and similarly for the other checkpoints. If the browser fails, use `fallback/fallback.pdf` and the trace tables below. The slides also have a PDF version. The separate closing-demo application is deferred; the final experiment block is fully teachable with the existing notebook and saved measurements.

00-09: applications, specification, and representation

Purpose: Establish why a route and a certificate of optimality are different outputs.

Say: "Finding a plausible route is easy to recognize. How do we know no cheaper route exists?"

Use vehicle route planning, robot/game movement, and network forwarding. Google Maps is a recognizable routing application, not evidence that its current production system uses the exact implementation in this lecture. For vehicle navigation, satellite positioning supplies a position; route planning is a separate problem. OSPF supplies a concrete documented Dijkstra application: shortest-path trees determine next hops and outgoing interfaces [R6]. Do not describe this as choosing an application port.

On slide 3 establish a finite directed graph, fixed additive costs, $c(u,v) \geq 0$, start s , and target t . Distinguish minimum cost, an actual route, and all single-source distances. Ask what should happen when t is unreachable or when $s=t$.

At minute 5, ask: "What is a node?" Intersections, grid cells, and navigable regions are modeling choices. Matrix, adjacency list, and neighbor generator are representation choices. With n vertices a matrix stores n squared entries; an adjacency list stores vertices plus actual edges. We use an explicit adjacency list. A grid can generate its neighbors without storing every edge.

Prompt for experienced students: "Does the current intersection describe the whole state when turning left has an extra cost?" Expected: perhaps not; include the incoming direction or edge. Do not turn this into a time-dependent routing lecture.

Exit condition by 09:00: Students can name the objective, the state/edge/cost model, and the concrete representation. Show `graphs/main_graph.py`, but leave its heuristic section hidden.

09-16: can we try every route?

Conjecture: Enumerate all routes and take the cheapest.

Ask students to total $S \rightarrow T$ and $S \rightarrow A \rightarrow B \rightarrow C \rightarrow T$: 15 and 8. Point at $A \rightarrow B \rightarrow A$: unrestricted walks can keep circling. Refine the proposal to simple paths. With nonnegative costs, removing a cycle never worsens a route, so some optimum is simple.

Open checkpoint 01. Ask why `on_path` must belong to the current recursive route, not globally ban every node seen by every candidate. Complete the guard and extension, then run it. The reference implementation counts 8 complete simple S -to- T paths and returns cost 8. It uses in-place backtracking and streams the incumbent instead of storing all routes. The shorter notebook version copies small path/set objects for readability; its memory behavior is not the reference $O(n)$ implementation.

Show the complete-graph rows in `results/comparison.md`. The count of paths can grow factorially [R5]. A budget-exhausted result is an unproven incumbent, not an optimal answer. Do not wait for an unbounded demonstration.

Transition at 16:00: "Different routes repeatedly reach the same state. Which prefixes could we throw away without losing an optimum?"

16-28: derive Dijkstra and test finalization

Conjecture: Keep only the cheapest discovered prefix to each state.

If two routes reach B at costs 6 and 3, the more expensive prefix is dominated under this model. Introduce $g(v)$ as the best discovered cost, not the unknown true optimum. Motivate relaxation: $\text{candidate} = g(u) + \text{weight}$; accept it only if it is strictly smaller than the current label. Then propose extracting the frontier node with minimum g [R1, R4].

Use the table below as the teacher's reference; write only changed labels on the board. Ties follow the listed neighbor order and a FIFO ticket issued at each insertion/improvement.

Selected	New labels after scanning outgoing edges	Observation
S, 0	A=2, B=6, D=2, T=15	Goal discovered, not certified
A, 2	B=3, C=8	B's old queue entry becomes obsolete
D, 2	E=3	A cheap detour competes with goal progress
B, 3	C=4, T=11	Update predecessors with labels
E, 3	F=4	No target information in priority
C, 4	T=8	Optimum discovered, still not extracted
F, 4	G=5	More cheap prefixes
G, 5	None	Candidate to T is 15
B, 6	Skip stale entry	Queued g differs from current g
C, 8	Skip stale entry	Older tie ticket than $T=8$
T, 8	Stop before scanning outgoing edges	Certificate under assumptions

Proof prompt, about 22:00: "A cheaper route to the selected u supposedly exists. Where does it first leave the finalized set?" Draw the settled region, predecessor x , and first outside node y . By induction x has its optimum; its relaxation exposed a candidate with $\text{dist}(y)=\text{delta}(y)$. Nonnegative suffix cost implies $\text{delta}(y)\leq\text{delta}(u)$. Minimum extraction gives $\text{dist}(u)\leq\text{dist}(y)$, contradicting a suboptimal selected label. Let students identify the inequality that uses the assumption.

Refutation, about 26:00: $S\rightarrow T$ costs 2, $S\rightarrow B$ costs 5, $B\rightarrow T$ costs -4. An early Dijkstra target pop would return 2, yet the optimum is 1. Ask precisely where the proof broke. The reference code rejects negative weights; this is a theorem counterexample, not a supported configuration.

Board record: "Finalize minimum g " $\rightarrow$ "Negative suffix reduces cost" $\rightarrow$ "Require nonnegative weights, or change algorithm."

Exit by 28:00: A student can explain why discovery is too early and why extraction is safe here. Continue to implementation; do not introduce Bellman-Ford beyond naming an alternative.

28-41: lazy Dijkstra, one necessary change at a time

28-34 / checkpoint 02. Fill candidate and the strict comparison. The queue holds route costs, not individual edge costs. Keep initialization and imports prewritten. Run the main graph and confirm cost 8.

34-38 / checkpoint 03. Pause after B enters at 6 and is improved to 3. Ask: "Can a simple heap efficiently find and replace the old B record?" Choose replacement insertion. Add `if popped_g != dist[u]: continue`. Python's standard heap supports this pattern without an indexed mapping [R2].

Show the lazy visualizer at its first stale event, not an entire second traversal. Switch the frontier from priority order to heap array to expose the distinction between a sorted explanatory view and real storage. The old B=6 and C=8 entries are skipped in the target-limited run. Later T records remain queued when the algorithm stops.

Update `parent[v] = u` on every strict improvement. Ask what goes wrong if parent changes only at first discovery. The initial parent of T is S, but the optimal route's predecessor of T is C. Recover the route backward only after the target cost is certified.

38-41 / checkpoint 04. Place the goal check after the non-stale pop. Show the isolated `stop_on_generation_bug`: it returns 15. Discuss target-only versus `target=None`. The latter continues until the heap is empty. Remaining tentative labels in an early-stopped run are not all shortest distances; only `Result.finalized` identifies Dijkstra's certified reachable labels.

Animation question: "Is this obsolete entry an incorrect route, or just a route that is no longer competitive?" Expected: it may be a real route; stale means superseded, not malformed.

Exit by 41:00: Cost 8, route S-A-B-C-T. Students can point to relaxation, the stale guard, parent update, and termination in the code.

41-48: eager Dijkstra and the cost of bookkeeping

Conjecture: The frontier needs only one live entry per node.

Show `IndexedMinPQ` as an existing component. Explain its heap array plus node-to-array-position dictionary. Every swap updates both positions. Fill checkpoint 05: decrease an existing priority or insert a newly reached node. Do not implement sift-up and sift-down during the seven-minute block.

Use one eager decrease-key event. Same shortest distances and search rule; different queue management. On the main graph lazy uses 13 insertions, 11 pops, 2 stale skips, and peak queue 6. Eager uses 9 insertions, 9 pops, 4 decreases, no stale skips, and peak queue 4. These exact counts are regression-tested.

Show slide 15's bounds. With adjacency lists, lazy binary-heap Dijkstra is $O(n+m \log(m+1))$; eager indexed-heap Dijkstra is $O((n+m) \log(n+1))$. For simple graphs their asymptotic times can agree. Smaller queue size does not prove smaller runtime; this Python indexed heap and the standard `heapq` have different constant costs. The empirical question remains open until measured [R7].

Mention d-ary and Fibonacci heaps only as extensions. The classical Fibonacci bound is $O(m+n \log n)$, not a universal current "state of the art" claim [R8]. Appendix slides carry details. If behind, skip the heap internals, never the next conceptual transition.

48-54: correct exploration can still be uninformed

Ask: "Which term in Dijkstra's priority mentions T?" None. Dijkstra respects directed edges, but its priority uses only the past cost, not estimated remaining cost.

Point at D-E-F-G. Each prefix is cheap, so Dijkstra processes it before the target cost 8 is certified. For the visual-wave intuition, use the supplied open-grid experiment or its static comparison results. All comparisons stop Dijkstra at a correct target extraction; do not handicap it by demanding all distances while A* stops early.

First repair attempt: Prioritize only h , estimated remaining cost. On the main graph, directly reachable T has $h(T)=0$. Greedy best-first search therefore selects the initially discovered cost-15 route, ignoring the accumulated cost. Ask: "What useful information did this repair throw away?" [R3]

Transition by 54:00: Propose adding the two quantities rather than replacing one with the other. Reveal h only now. The main graph uses $h=|x-6|+|y|$ from the supplied coordinates, not arbitrary values painted beside nodes.

54-61: derive A* and the lower-bound promise

Write $f(v)=g(v)+h(v)$. Trace S, A, B, C, T. Their selected (g,f) values are (0,6), (2,7), (3,7), (4,7), and (8,8). D remains queued at $g=2$, $h=7$, $f=9$. A* processes four adjacency lists instead of eight, returning the same cost.

Ask: "Why should these estimates preserve optimality?" Change $h(A)$ to 20 in the overestimate preset. The actual code then returns cost 11 instead of 8: the optimal prefix was hidden behind inflated priorities. Introduce admissibility, $0 \leq h(v) \leq h^*(v)$ and $h(T)=0$ [R3].

The frontier-bound argument is not that every f -value bounds the global optimum. An optimal-prefix frontier representative has $f \leq C^*$. A suboptimal goal has $f=g > C^*$. It cannot be minimum while that representative is available. Highlight the unfinished proof obligation: what if a state on the optimal route was expanded earlier at a worse cost and never allowed back?

Expected misconception: "If h is smaller it is always faster." No. A valid but weak h can do more search, and even a more informative heuristic has evaluation cost. Save runtime claims for the experiments.

61-68: admissibility is not permission to close forever

Use the separate four-node graph. It is intentionally smaller than the main graph so students can inspect the failure without bookkeeping overload.

$S \rightarrow A: 3$	$S \rightarrow B: 1$	$B \rightarrow A: 1$	$A \rightarrow T: 3$
$h(S)=0$	$h(A)=0$	$h(B)=4$	$h(T)=0$

Ask the class to confirm admissibility. From B the true remaining cost is 4; $h(B)$ is exact. A^* first expands A at $g=3, f=3$, discovering T at 6. B has $g=1, f=5$ and is expanded next. Its edge to A yields $g(A)=2$. If A is permanently closed, cost 6 survives; if reopened, it improves T to 5.

Use the visualizer's **Broken: never reopen** control only for this example. Ask students to predict the outcome before toggling. The two source views are actual, separate Python functions. Keep the wrong variant visibly labeled.

Repair 1: Re-enqueue and re-expand when a genuinely smaller g is found. This is not the same as discarding obsolete queue records. **Repair 2:** Prove the stronger condition that removes the need for reopening: $h(u) \leq c(u,v) + h(v)$ for every edge.

At $B \rightarrow A$ the condition fails: $4 > 1 + 0$. Interpret it as a heuristic drop larger than the edge cost. Then use the PhD-level connection:

$$\begin{aligned} c_h(u,v) &= c(u,v) + h(v) - h(u) \\ g_h(v) &= g(v) + h(v) - h(S) \end{aligned}$$

Consistency means nonnegative reweighted edges. A^* is Dijkstra ordering in this reweighted graph, with a constant offset [R4]. In the example $B \rightarrow A$ becomes -3: the finalization problem has reappeared as a negative edge. Leave a full telescoping calculation in the appendix if time is tight.

Exit by 68:00: Students distinguish admissibility, consistency, and reopening. Do not end this block with the unqualified statement "A* never revisits a node."

68-77: implement A* and state the right bound

Complete checkpoint 06. Change the priority to $\text{candidate} + h[v]$; carry the queued g separately. Keep the stale test on g . Do not add a permanent closed set to the reference algorithm. Its expanded set records statistics only.

Run the main graph, the reopening example, and $h=0$. With $h=0$ the same tie policy reproduces lazy Dijkstra's route and queue statistics. In the inconsistent example the correct implementation reports one re-expansion and cost 5. The broken variant reports cost 6.

A consistent constant-time heuristic gives the same explicit-graph worst-case order as lazy Dijkstra, potentially with less explored work. Under admissible inconsistency the one-expansion-per-node analysis no longer applies. Distinguish n and m from the branching-factor/depth description of an implicit state space.

Prompt: "Did we change only a line of code?" Syntactically nearly; logically the guarantee now depends on what h means and how improved routes are handled.

77-86: experiments without the deferred closing application

This release intentionally contains no street-map, game, maze-animation, or Three.js closing-demo application. The same nine-minute teaching objective is supported by the solution notebook and saved `results/comparison.md`.

77-79 / scaling. Show complete-graph rows. Explain `budget_exhausted` versus `not_run_size_limit`. Do not compare the time of an incomplete search to a completed optimum as if they solved the same task. Ask whether eager Dijkstra actually beat lazy on these implementations; read the measurements, not the algorithm name.

79-83 / heuristic quality. Run the notebook's grid experiment with zero, Euclidean, and Manhattan h on the same four-neighbor unit grid with a wall. Compare returned cost and expansions. Then use the open-grid rows. Ask why $h=0$ should match Dijkstra, and why a wall-free relaxation remains a lower bound when walls are added.

83-85 / change the rules. Offer diagonal movement costing one. For a diagonally adjacent goal, Manhattan gives 2 while the true cost is 1. Let students repair the heuristic (Chebyshev distance on the unobstructed eight-neighbor unit grid) rather than calling the original heuristic "bad" in isolation.

85-86 / interpret the data. Check that every completed valid search returns the same optimum. Distinguish state expansions, queue memory, and elapsed runtime. Instrumented search timing excludes rendering and traces but includes initialization, counters, and path recovery. Runtime is machine-specific. The supplied comparison does not establish universal performance rankings.

When the separate closing application is built, it can replace these visual forms without changing the questions, graph model, counters, or validity checks.

86-90: synthesis and exit exercise

Ask students to answer without looking at the summary first:

1. Why is first discovery of T insufficient? What makes a later stopping point valid?
2. An admissible heuristic is used. May every expanded state be permanently closed? What additional condition or implementation repair is needed?
3. Are stale-entry rejection and reopening contradictory operations?

Expected answers: A discovered route is only an upper bound; a valid non-stale goal extraction is certified under the algorithm's assumptions. Admissibility alone does not justify permanent closure: consistency is a sufficient strengthening, or allow improvements to propagate through reopening. Stale rejection discards superseded records; reopening propagates a newly improved route after an earlier expansion.

Finish with the progression: **enumerate routes -> discard dominated prefixes -> prove finalization -> add goal information -> test the estimate and the finalization rule**. The mathematical reason for a line of code is the intended learning outcome.

Rehearsal checks and contingency decisions

A class that already knows both algorithms. Emphasize the proof line using nonnegativity, inconsistent admissible h , potential reweighting, and whether the state representation is sufficient. Do not spend the saved time listing more algorithms.

A class with uneven Python knowledge. Ask students for expressions and invariants; type them yourself. Use the notebook solutions after one failed execution. Participation need not mean every student configures a laptop.

Five minutes behind at minute 41. Show the eager code diff and a single decrease-key snapshot; skip all heap internals and the extension discussion. Preserve A*'s 54-minute start.

Five minutes behind at minute 68. Show the completed A* code with the three changed concepts highlighted, then run the reopening test. Use only one comparison fixture in the final experiments.

Browser or projector failure. `fallback/fallback.pdf` includes the key queue states; the full PowerPoint has a PDF counterpart. The main graph and this guide's tables permit the core lecture to continue on the board.

Do not conflate: fewest hops and minimum weight; cheapest edge and cheapest accumulated route; model and storage; discovered and finalized; a heap and a fully sorted sequence; an admissible heuristic and a consistent one; a stale record and a genuinely improved state; measured runtime and an asymptotic bound; a partial incumbent and a certified optimum.

Source and implementation pointers

`docs/technical_notes.md` contains full proofs, conventions, numerical caveats, and complexity arguments. `docs/references.md` resolves [R1]–[R8], [P1], and [P2]. `tests/test_algorithms.py` checks the displayed numerical examples, invalid inputs, zero-cost cycles, equality cases, all-distances mode, indexed-heap invariants, and random graphs against an independent oracle.

The slide appendix is optional. It contains a more complete proof, potential reweighting, complexity details, heap extensions, and references. It is not included in the 90-minute schedule.